Lab4 Advanced: 7-Segment Display Snake Game

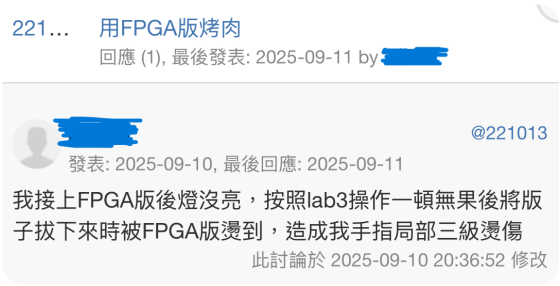
Submission Due Dates:

Demo:	2026/10/27 17:20
Source Code:	2026/10/27 18:30
Report:	2026/11/01 23:59

1. Project Objective and Overview

This assignment guides you through designing a hardware-based Snake game using SystemVerilog. You will utilize a 4-digit 7-segment display as the physical game board, and react to user inputs managed by a Finite State Machine (FSM).

This particular snake has an unusual passion: it loves to use the warm FPGA board to grill its favorite meat! Your mission is to design this game so that the snake can relentlessly pursue its culinary love across the display segments.



2. System I/O Interface Definition

The core game module must implement the following input and output ports:

I/O Signals

Signal	Connected to	Description
`clk`	W5	Clock signal with the frequency of 100 MHz
`rst`	BTNC	Asynchronous active-high reset
`led[15:0]`		Volume level and snake length
`digit[0:3]`		7-segment digit enable
`display[6:0]`		7-segment display control
`PS2_DATA`		Keyboard data
`PS2_CLK`		Keyboard clock
`Pmod JB 1~6`		Pmod I2S

3. Finite State Machine (FSM) Lifecycle and Game Mechanics

The game is controlled by a 3-state FSM: **INITIAL**, **PLAY**, and **FINAL**.

Reset Behavior

`rst` (asynchronous, active-high) **immediately** clears the FSM to INITIAL and re-initializes all internal registers, regardless of the current state.

Global Configurations (all states)

Keyboard Input Handling

- **Direction Control**: Use the **Arrow Keys** on the PS/2 keyboard.
- **A key press should only register once** (no repeats while holding).
- **When a key is held down, no other key should activate until it is released.**

Volume Control System

- **Volume Control**: Use the keyboard's **'W' (Volume Up)** and **'S' (Volume Down)** keys to adjust the volume. The volume has three levels, with **Level 2** as the default. The amplitude at each level must remain within the range of `-0x2000` to `+0x2000`.
  - **Hint**: You can implement this feature based on the provided `note_gen.v`.
  - **Requirement**: The three levels must be heard and distinguishable. Pressing Volume Up at level 3 or pressing Volume Down at level 1 has no effect.
- **Volume Display**: Use `led[15:13]` to display the current volume level according to the table below:

Volume Level	`led[15]`	`led[14]`	`led[13]`
Level 1	0	0	1
Level 2 (Default)	0	1	1
Level 3	1	1	1

Score Display (Snake Length)

- The current length of the snake acts as the player's score and is continuously displayed on the LEDs across all states.
- **The display format works by lighting up a number of LEDs equal to the decimal digit value.** (i.e., if the digit is **N**, then rightmost **N** LEDs should be turned on).
- `led[12]` and `led[9]` are always 0, serving as separators.
- `led[11:10]` displays the tens digit of the snake's length.
- `led[8:0]` displays the ones digit of the snake's length.
- **Example**: If the snake's length is **18**:
  - The tens digit is 1 (so rightmost 1 LED is lit): `led[11:10] = 2'b01`.

- The ones digit is 8 (so rightmost 8 LEDs are lit): `led[8:0] = 9'b0_1111_1111`.

LFSR (Random Number Generator, design by yourself)

- The LFSR is free-running and **updates every `clk` cycle**.
- **Size Requirement**: The LFSR must be at least **5 bits** wide.
- **State Requirement**: The lowest 5 bits (`lfsr[4:0]`) must be able to cycle through at least **17 distinct states** (values) to ensure sufficient randomness for meat generation.
- The lowest 5-bit output is mapped to a candidate of meat location.

State INITIAL

Behavior: The system waits in standby mode. The LFSR runs continuously in the background.

I/O States:

- `7-segment`: Digit 0,1 is dark, digit 2,3 shows a captured LFSR lowest 5 bits value in decimal, **with an initial display value of 00**. (The displayed value updates to the current LFSR lowest 5 bits value when `Space` is pressed).
- **Audio Feedback**: Play **C5-D5-E5-F5-X-X** in a continuous loop. Each note's duration is `2^23 / 100M` seconds, where X represents silence.

Setup:

- Initial length: **3 nodes**
- Initial direction: **Right**

Initial snake location:

Part	Location
Tail	Digit 0, segment a
Body	Digit 1, segment a
Head	Digit 2, segment a

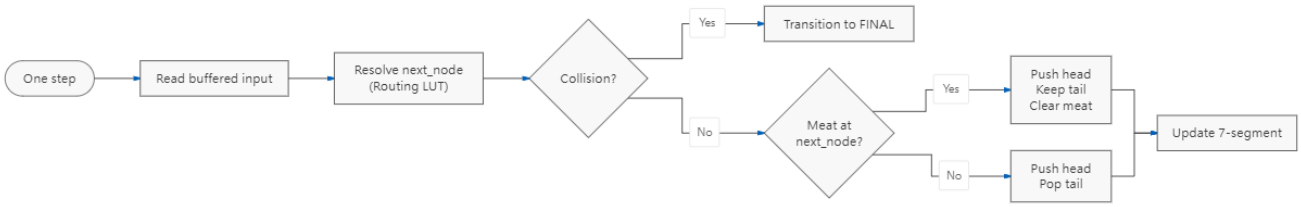
Transition: Use the `Enter` key to move to the **PLAY** state.

State PLAY

Behavior: The snake moves forward continuously. At each movement `step` (every **2^26 clock cycles**), the following sequence occurs:

1. Read the buffered player input (if any).
2. Resolve the next node via the Routing LUT.
3. Check for collision (self-bite or dead end).
4. If collision → transition to FINAL.
5. If no collision → check for meat and update snake body.

Per-Step Update



I/O States:

- `7-segment`:
  - Displays the snake body, the segments where the snake occupies are lit up, others are dark.
  - The meat segment blinks twice during each movement `step` (ON → OFF → ON → OFF). Each ON or OFF interval lasts one quarter of a movement `step`.
  - You **don't** need to implement the brightness gradient for the snake body shown in the demo video.
- **Audio Feedback**:
  - **Eat Meat**: When the snake successfully eats meat, play a short, high-pitched tone, such as **G5 (784 Hz)**. Duration: `2^23 / 100M` seconds.
  - **Silence**: When no meat is eaten, no audio is played.

Meat Generation:

- **At most one meat** exists on the board.
- **Collision-Free Spawning**: When meat needs to be spawned (no meat on board), the system checks the current candidate of meat location (using `lfsr[4:0]` e.g., `lfsr[4:0] - 1`. You can design the relation between `lfsr[4:0]` and meat location freely, as long as randomness is ensured). The candidate must be valid (seg a-g), and it must not overlap with the snake. If invalid, the system waits for the next `clk` (since the LFSR continuously updates) to check the next candidate.

Death Conditions (Transitions to FINAL **in next step**):

- **Self-bite**: next location of snake is occupied by the snake, **except** when the next location is the current tail position AND **the tail will be popped** (no meat eaten).
- **Dead end / Hard wall**: next location of snake is not on the 7-segment display.

Hint: Snake Body Tracking (FIFO)

Store the snake's body in an **ordered sequence** of Node IDs in a FIFO (first-in first-out) queue.

- **Head pointer**: Points to the most recently pushed node.
- **Tail pointer**: Points to the oldest node.
- **FIFO depth**: 32 entries recommended (power of 2 allows natural 5-bit pointer wrapping).

Movement Update (update pointer):

- **Push**: The new head node is pushed onto the FIFO (i.e., the head pointer is incremented).

- **Pop (if no meat eaten)**: The tail node is popped from the FIFO (i.e., the tail pointer is incremented).
- **No Pop (if meat eaten)**: The tail remains. The snake grows by 1 node.

State FINAL

Behavior: Show score blinking animation and play a descending scale.

I/O States:

- `7-segment`:
  - **Visual Animation**: Display the score using the following blinking animation: Digits 0,1 and Digits 2,3 alternately display the score (snake length). The switching interval is `2^25 / 100M` seconds. When one pair (e.g., Digits 0,1) is displaying the score, the other pair (Digits 2,3) must be completely dark.
- **Audio Feedback**:
  - Play **E4-D4-C4** sequentially once upon entering the state. Each note's duration is `2^24 / 100M` seconds. (This can be played alongside the visual score animation).
  - **Silence**: After the sequence is played, maintain **silence** for the remainder of the state.

Transition: Use the `Enter` key or `Space` bar to move to the **INITIAL** state.

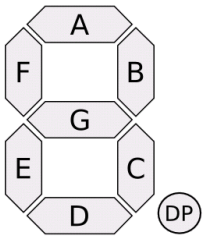
Frequency Reference Table:

Note	C4	D4	E4	F4	G4	A4	B4
Hz	262	294	330	350	392	440	494
Note	**C5**	**D5**	**E5**	**F5**	**G5**	**A5**	**B5**
Hz	523	587	659	699	784	880	988

---

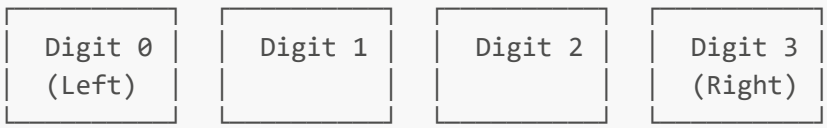
4. Coordinate System and Topology

Single-Digit Topology



Source: Wikipedia

Digit Physical Layout



Digits are numbered 0 (leftmost) to 3 (rightmost) on the physical display.

---

5. Core Transition and Routing Behavior

Because the board is a graph, moving "Up" or "Right" behaves differently depending on the exact segment the snake's head currently occupies. This section fully defines every valid movement path.

A. Segment Classification and Valid Turns

Segments are classified into two types:

- **Horizontal Segments (a, d, g)**: The snake travels left or right along these. Valid player inputs are generally **Up** or **Down** to turn onto a vertical segment; however, **steering directly into a wall (e.g., trying to turn Up while head is on segment 'a', or Down on segment 'd') is an invalid input and will be ignored**. For the detailed definition of all valid inputs, please refer directly to **Section 5B**.
- **Vertical Segments (b, c, e, f)**: The snake travels up or down along these. Valid player inputs are **Left** or **Right** to turn onto a horizontal segment.

> No input and an invalid input have the same effect: the snake continues moving according to its current direction. An input is invalid if it matches the segment's orientation (e.g., pressing Left/Right while head is on a horizontal segment) or attempts to steer into an outer wall.

B. Complete Routing Lookup Table (LUT)

The following tables fully define the next segment and next digit for every combination of current segment, player input, and direction state.

Horizontal Segments (a, d, g)

Current	Valid Input	Next Seg	Next Digit	Direction
a	Down	**b**	same	Right
a	Down	**f**	same	Left
a	*(none / invalid)*	**a**	digit + 1	Right
a	*(none / invalid)*	**a**	digit − 1	Left
d	Up	**c**	same	Right
d	Up	**e**	same	Left

Current	Valid Input	Next Seg	Next Digit	Direction
d	*(none / invalid)*	**d**	digit + 1	Right
d	*(none / invalid)*	**d**	digit − 1	Left
g	Up	**b**	same	Right
g	Up	**f**	same	Left
g	Down	**c**	same	Right
g	Down	**e**	same	Left
g	*(none / invalid)*	**g**	digit + 1	Right
g	*(none / invalid)*	**g**	digit − 1	Left

> **On horizontal segments, Up and Down inputs also update the `direction` register** to the new input value.

Vertical Segments (b, c, e, f)

Current	Valid Input	Next Seg	Next Digit	Direction
b	Right	**a**	digit + 1	Up
b	Right	**g**	digit + 1	Down
b	Left	**a**	same	Up
b	Left	**g**	same	Down
b	*(none, invalid)*	⚠ (collision)	same	Up
b	*(none, invalid)*	**c**	same	Down
c	Right	**g**	digit + 1	Up
c	Right	**d**	digit + 1	Down
c	Left	**g**	same	Up
c	Left	**d**	same	Down
c	*(none, invalid)*	**b**	same	Up
c	*(none, invalid)*	⚠ (collision)	same	Down
f	Right	**a**	same	Up
f	Right	**g**	same	Down
f	Left	**a**	digit − 1	Up
f	Left	**g**	digit − 1	Down
f	*(none, invalid)*	⚠ (collision)	same	Up

Current	Valid Input	Next Seg	Next Digit	Direction
f	*(none, invalid)*	**e**	same	Down
e	Right	**g**	same	Up
e	Right	**d**	same	Down
e	Left	**g**	digit − 1	Up
e	Left	**d**	digit − 1	Down
e	*(none, invalid)*	**f**	same	Up
e	*(none, invalid)*	⚠ (collision)	same	Down

> **On vertical segments, Left and Right inputs also update the `direction` register** to the new input value.

> ⚠ **Collision = Dead End**: When the snake hits a hard wall, the routing logic encounters a dead end. You need to implement logic to detect this condition and trigger a collision transition to FINAL.

C. Horizontal Wrap-Around

Horizontal movement wraps around across the four digits. The digit index is a 2-bit value; natural overflow handles wrap-around:

```
Moving Right:  digit index increments  (... → 2 → 3 → 0 → 1 → ...)
Moving Left:   digit index decrements  (... → 1 → 0 → 3 → 2 → ...)
```

Specifically, when the snake exits a vertical segment at the edge of a digit:

Exit From	Input	Entry To	Target Digit
b or c (right side)	Right	a or g or d	digit + 1 (with wrap)
f or e (left side)	Left	a or g or d	digit − 1 (with wrap)

The exact target segment (whether it enters the upper or lower horizontal) is determined by the Routing LUT above.

D. Directional Inertia, 180° Prevention, and Input Sampling

Direction Register

The `direction` register is updated **only** when a valid turn occurs (i.e., a perpendicular input on the appropriate segment type). It is **not** updated on straight-line movement.

Directional Inertia

If no valid input is received before the next `step`, the snake automatically continues along its current path based on the `direction` register and the Routing LUT.

180° Prevention

The segment topology naturally prevents 180° reversal:

- On horizontal segments, only Up/Down inputs are accepted (perpendicular).
- On vertical segments, only Left/Right inputs are accepted (perpendicular).

Since the snake can only turn 90°, it is physically impossible to reverse direction in a single step.

Input Sampling Rule

If multiple keyboard inputs occur in one `step`, **only the last input is retained**. All previous inputs are overwritten. At each `step`, the buffered input is consumed and the input buffer is cleared.

E. Snake Self-Bite and Crossing Examples

Note: Because of the tail-pop exception in the Self-bite rule, **the body crossing paths**, or **the head moving directly into the tail's current position (regardless of the tail's movement direction)**, are allowed and do **not** count as a collision, provided that the tail is popped in the same movement step.

Example 1: Chasing the Tail (Safe vs Self-bite)

The head moves into the exact segment the tail is currently occupying. Because the tail vacates the segment in the same clock cycle, no collision occurs.

e.g. Current sequence (Tail to Head): f -> g -> c -> d -> e



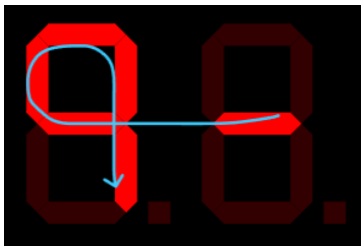
The Head is at 'e', currently moving Up.

- Straight: Next node is 'f'. Since 'f' is the Tail and will be popped this step, it is SAFE.
- Left: Next node is 'g' on the left digit (empty). SAFE.
- Right: Next node is 'g' on the same digit. Since 'g' is a body part and will NOT be popped, this is a Self-bite COLLISION!

Example 2: Body Crossing (Safe)

The snake can visually "cross" its own overall path. As long as it steps onto a segment that is currently empty (or vacated by the tail), it is safe.

e.g. Current sequence (Tail to Head, format: digit-segment): 1g -> 0g -> 0f -> 0a -> 0b -> 0c



Here, the snake crosses through the horizontal center ('0g') from Digit 1 and loops around the top half of Digit 0. Since each segment is a distinct node, this visual crossing is perfectly valid and safe.

F. Play State Example Walkthrough

step	snake (tail → head)	meat	direction	input	explanation
0	0a → 1a → 2a	1g	right	no input	no input, the snake moves in the current direction
1	1a → 2a → 3a	1g	right	left	invalid input, the snake moves in the current direction
2	2a → 3a → 0a	1g	right	no input	#5-C. Horizontal Wrap-Around, the snake moves from 3a to 0a
3	3a → 0a → 1a	1g	right	down	turn down
4	0a → 1a → 1b	1g	down	right, left	the input buffer keeps only the last input
5	0a → 1a → 1b → 1g	2a	left	up	the meat is eaten, the snake grows by 1 node, so the tail is not popped and new meat is generated at a new position in several clock cycles
6	1a → 1b → 1g → 1f	2a	up	right	
7	1b → 1g → 1f → 1a	2a	right	no input	although the snake moves to its previous position, it is safe since the previous position is the tail and it didn't eat meat in this step
8	1b → 1g → 1f → 1a → 2a	2g	right	no input	the snake grows by 1 node & new meat appears
14A	2b → 2g → 1g → 1f → 1a → 1b	3c	down	no input	
15A	2g → 1g → 1f → 1a → 1b → 1c	3c	down	no input	#5-E-Example 2: Body Crossing (Safe)
16A	dead				the snake hits wall

step	snake (tail → head)	meat	direction	input	explanation
14B	2b → 2g → 1g → 1f → 1a → 1b	3c	down	left	
15B	dead				the snake hits its body part 1g, the next position of head in 14B is occupied by the snake itself

Grading

GLOBAL	INITIAL	PLAY	FINAL
25%	15%	45%	15%

Questions and Discussion

Please answer the following questions in your report. Be clear and concise in explaining the design principles. Please respond these questions in your own words. Plain language is fine.

1. In the template (lines 154–155), `snake_body` and the pointers form a queue. Because hardware supports parallel operations, a queue can also be implemented using only one pointer. **Write pseudo code to implement a queue** (with operations: `pop`, `push`) **using only a pointer** (e.g., `end_ptr`). Assume that `pop` and `push` occur only when they are valid; you do not need to handle queue underflow or overflow. (**Note:** When updating registers, the signal width must be fixed. A variable-length assignment such as `queue[0:n] = something` is not allowed. Any `for` loop must have bounds that are fixed at synthesis time, such as constants or parameters. The bounds must not depend on runtime signals such as `end_ptr`.) (Adapted from 2025 Mid 2, Part E)

2. How do you control and record the snake's position in the grid, and how are the active states of the 28 display segments managed (e.g., stored in registers vs. computed via a LUT)? Explain your architectural choices.

3. How do you design the audio control logic to handle both continuous looping melodies (in the INITIAL state) and single-triggered sound effects (like eating meat in the PLAY state or the game over sequence in the FINAL state)?

4. If you were to implement a brightness gradient for the snake body (e.g., the head is the brightest and the tail is the dimmest, shown in demo video), how would you design it? You can briefly describe the data structure you would use to store each segment's brightness level, how you would update these values when the snake moves, and how the brightness level would be translated into a visible intensity difference on the 7-segment display.

Report Guidelines

Your report should include, but are not limited to, the following items. Prepare your report using the provided Advanced Lab Report Template. In addition to the general requirements in the template, answer all questions

and explain your design.

A. Architecture / Block Diagrams (30%)

B. Questions & Discussions (20%)

C. Discussion & Analysis (40%)

D. Additional Work (Extra Credit: up to 10%)

Attention

- Do not modify `KeyboardDecoder.sv`. Otherwise, you will not receive full credit.
- DO NOT copy-and-paste code segments from the PDF materials to compose your design. It may also paste invisible non-ASCII characters, leading to hard-to-debug syntax errors.
- Hand in your SystemVerilog files. Merge all modules into a single `.sv` file.
- You may use the provided template modules. Do not include the unmodified template modules in your submission: `keyboard_decoder`, `KeyboardCtrl`, `Ps2Interface`, `one_pulse`, `speaker_control`, `music_example`, `player_control` and `debounce`. Include any provided module that you modify.
- DO NOT submit compressed ZIP files, which will be considered an incorrect format.
- Report filename format: `lab4_report_{StudentID}.pdf` (e.g., `lab4_report_123456789.pdf`).
- You have the responsibility to check if the submission is successful.
- Prepare bitstream files before demo to avoid delays.
- Be ready to answer TA questions during demo.
- Post questions on EECLASS forum if unclear.
- Display the digits according to lecture material:

